Heap

This document will be compiled incrementally.

Almost Complete Binary Tree

A **binary tree** ("binary" means every node can have at most two children) is said to be an **almost complete binary tree** when:

- **All levels except the last are completely filled**
  Every level of the tree (from the root to the level just before the last) has the maximum number of nodes possible for that level.

- **The last level is filled from left to right**
  Nodes at the last level are arranged as far to the left as possible, with no gaps between them.

Why this matters

The **heap data structure** is internally maintained as an **almost complete binary tree**. This structure:

- Guarantees a compact shape
- Allows efficient array-based representation
- Enables `O(log n)` insertion and deletion operations

Heap and Priority Queue

A **heap** is most often used in an abstracted form called a **Priority Queue**, which is a very important data structure.

The key advantage of a priority queue is that we can always access the **highest-priority** or **lowest-priority** element efficiently.

When we need to process elements based on a specific priority (for example, highest value, lowest value, earliest deadline, etc.), we construct a heap using that **priority-defining property as the key**.

Operations and Time Complexity

- **Insert**: `O(log n)`
- **Lookup (peek min / max)**: `O(1)`
- **Deletion (remove min / max)**: `O(log n)`

Why this is powerful

This combination of fast access and efficient updates makes heaps / priority queues ideal for:

- Scheduling problems
- Dijkstra's and Prim's algorithms
- Top-K problems
- Real-time systems where priority matters

Heap Property

The **heap property** defines the relationship between a parent node and its child nodes in a heap. This property ensures that the heap remains ordered in a way that supports efficient operations.

The exact relationship depends on the **type of heap** being used. There are two main types:

Min-Heap

- The value of each **parent node is less than or equal to** the values of its child nodes.
- As a result, the **smallest element is always at the root** of the heap.

Max-Heap

- The value of each **parent node is greater than or equal to** the values of its child nodes.
- As a result, the **largest element is always at the root** of the heap.

Key takeaway

The heap property does **not** impose a global ordering like a BST. It only guarantees ordering **between parent and children**, which is sufficient to efficiently retrieve the minimum or maximum element.

Key Points about the Heap Property

- **Local Relationship**
  The heap property only enforces ordering between a parent node and its immediate children. It does **not** impose any ordering between sibling nodes or across different levels of the tree.

- **Preservation During Operations**
  During insertion or deletion, the heap property may be temporarily violated. A process called **heapification** restores the property by repositioning elements until the heap property is satisfied again.

Array Representation of Heap

Heaps are efficiently represented using an **array**, where the **root node is stored at index** `0`. Because a heap is an almost complete binary tree, this array representation is compact and does not waste space.

The relationship between parent and child nodes is defined using simple mathematical formulas:

- **Left child of node at index** `i`:
  `2 * i + 1`
- **Right child of node at index** `i`:
  `2 * i + 2`
- **Parent of node at index** `i`:
  `(i - 1) / 2` (integer division)

Why this works

Because the heap is an almost complete binary tree, nodes are filled level by level from left to right. This predictable structure allows us to compute parent–child relationships directly from indices without using pointers.

Why Array is Preferred Over Pointer-Based Tree Structure

Memory Efficiency

An array-based representation does not need to store explicit pointers to the left child, right child, or parent.

In a pointer-based tree, each node typically requires **three extra references** (left, right, parent). In contrast, an array computes these relationships mathematically using indices, resulting in **significant memory savings**, especially for large heaps.

Cache Locality

Arrays store elements **contiguously in memory**, which leads to much better cache performance.

- When accessing a parent node, its children are usually located nearby in memory.
- This reduces cache misses and improves real-world performance.

In pointer-based trees, nodes may be scattered across memory, leading to frequent cache misses.

Simplicity

- No pointer management is required during insertion or deletion.
- The heap structure is preserved by simply appending/removing elements at the end of the array and applying **heapification**.
- This makes implementations simpler, safer, and less error-prone.

Bottom line

Heaps are about **performance and predictability**, and array-based implementations win decisively over pointer-based trees in both aspects.

Why This Relationship Is Always True

Visual representation of heap indexing: for a node at index `i`*, left child is* `2i+1`*, right child is* `2i+2`*, and parent is* `(i-1)/2` *(integer division).*

The index-based parent–child relationship in a heap works because an **almost complete binary tree** is filled **level by level, from left to right**, without gaps.

Level-wise indexing

When nodes are numbered sequentially starting from `0`:

- **Level 0 (root)**: 1 node
  Index: `0`
- **Level 1**: 2 nodes
  Indices: `1, 2`
- **Level 2**: 4 nodes
  Indices: `3, 4, 5, 6`
- **Level 3**: 8 nodes
  Indices: `7, 8, 9, 10, 11, 12, 13, 14`

Each level contains **exactly twice** as many nodes as the previous one.

Parent–child index relationship

For any node at index `i`:

- Its **left child** is placed at the first available position of the next level segment:
  `2 * i + 1`
- Its **right child** is placed immediately after that:
  `2 * i + 2`

This happens deterministically because:

- Nodes are filled left to right
- There are no gaps in the array
- Each level is contiguous in memory

Core reason

This relationship is a direct consequence of:

- The **binary tree structure** (each node has at most two children)
- The **almost complete property** (no missing nodes except possibly at the far right of the last level)

Together, these guarantees make the array index formulas **always valid** for heaps.
